

Attempt any six (including subparts)

1 a) Write a tail-recursive algorithm (using C pseudocode) which takes the value of n from the user as input and displays the n^{th} term of the Fibonacci series as output. Specify its asymptotic time complexity (using smallest upper bound).

b) A complex-valued matrix X is represented by a pair of matrices (A, B) , where A and B contain real values. Write an algorithm (using C pseudocode) that computes the product of two complex-valued matrices (A, B) and (C, D) , where $(A, B) * (C, D) = (A+iB) * (C+iD) = (AC-BD) + i(AD+BC)$. Determine the asymptotic time complexity (using smallest upper bound) of your algorithm if the matrices are all of dimensions $n \times n$.

c) Explain with suitable examples 0-based indexing, 1-based indexing and n -based indexing for arrays.

(3+4+3)

2 Write down algorithms (using C pseudocode) to perform the following operations on a stack (implemented using array) of size n (where the value of n is provided by the user as input) and specify their asymptotic time complexities (using smallest upper bound).

- Push k number of elements (provided by the user as input) to the stack (where the value of k is also taken from the user as input)
- Reverse the elements of the given stack using recursion / the call stack only.
- Display the contents of the stack

(2+6+2)

3 a) Let an array be represented as $A(2:20, 3:30, 1:25, 0:25, 5:30)$. Assuming column major representation of the array (using sequential byte addressing scheme) with 2 bytes per element. If k is the address of $A(4,5,6,7,5)$ then the address of the element represented as $A(5,10,15,6,12)$ is _____ [Fill in the blank correctly].

b) For the following, find the smallest function $g(x)$ for which $f(x) = O(g(x))$

i) $f(x) = 2 + 6x + 18x^2/2! + 54x^3/3! \dots + \infty$

ii) $f(x) = 1 + x + x^2/2! + x^3/3! + \dots + x^n/n!$

- c) For the following function [where, $\text{end_index} - \text{start_index} = n-1$], using asymptotic analysis, the time complexity of **foo** (using smallest upper bound) is _____ and the recursion stack space required by **foo** (using smallest upper bound) is _____

```
int foo(int array[], int start_index, int end_index, int element){
    if (end_index >= start_index){
        int middle = start_index + (end_index - start_index )/2;
        if (array[middle] == element)
            return middle;
        if (array[middle] > element)
            return foo(array, start_index, middle-1, element);
        return foo(array, middle+1, end_index, element);
    }
    return -1;
}
```

- d) Write an algorithm (using *C* pseudocode) to print the number of occurrences of each character in a given string (provided by the user as input) of length *n*. Determine the asymptotic time complexity (using smallest upper bound) of your algorithm.

(3+2+2+3)

- ~~4a)~~ Write an algorithm (using *C* pseudocode) to perform the addition of two polynomials [where each polynomial is stored the array as a list of elements in the form (*m*, e_{m-1} , b_{m-1} , e_{m-2} , b_{m-2} , ..., e_0 , b_0) where, the first entry *m* represents the number of non-zero terms followed by two entries (e_i , b_i) for each non-zero term (representing an exponent-coefficient pair) in the order $e_{m-1} > e_{m-2} > \dots > e_0 \geq 0$] and print the resultant polynomial. Specify the asymptotic space and time complexity (using smallest upper bound) of your algorithm.

- b) Explain what is understood by a priority queue. What is the difference between direct and indirect recursion ? What is tail recursion and what is its advantage ?

- c) Prove that if $f(n) = a_m n^m + \dots + a_1 n + a_0$, the $f(n) = O(n^m)$, for $n \geq 1$, where $a_0, a_1, \dots, a_m$ are constants.

(4+3+3)

5. Write down algorithms (using C pseudocode) to perform the following operations on a non-circular queue (implemented using array) of size n (where the value of n is provided by the user as input) and specify their asymptotic time complexities (using smallest upper bound).

- Insert k number of elements (provided by the user as input) to the non-circular queue (where the value of k is also taken from the user as input).
- Reverse the elements of the non-circular queue using stack
- Display the contents of the circular queue

non

(2+6+2)

6 a) For the following function the best case asymptotic time complexity (using smallest upper bound) is _____ and the worst case asymptotic time complexity (using smallest upper bound) is _____

```
void myfunc(int array[], int size) {
    for (int step = 1; step < size; step++) { - size + 1
        int key = array[step];
        int j = step - 1;
        while (key < array[j] && j >= 0) {
            array[j + 1] = array[j];
            --j;
        }
        array[j + 1] = key;
    }
}
```

b) Write algorithms (using C pseudocode) for inserting and deleting an element (provided as input from the user) to the priority queue (implemented using array) of size n (where the value of n is provided by the user as input). Specify their asymptotic time complexities (using smallest upper bound).

c) Show whether the following are true or false with suitable proof/ justification.

- | | |
|---------------------------------|--|
| i) $6 \cdot 2^n + n^2 = O(2^n)$ | ii) $6 \cdot 2^n + n^2 = O(n^{10000})$ |
| iii) $10n^2 + 4n + 2 = O(2^n)$ | iv) $10n^2 + 4n + 2 = O(n^{2.5})$ |
| v) $10n^2 + 4n + 2 = O(n^2)$ | vi) $10n^2 + 4n + 2 = O(n)$ |
| vii) $n/500000 = O(500000)$ | viii) $500000 = O(5)$ |

(2+4+4)

- 7a) Write your recursive algorithms (using *C* pseudocode) to find the Greatest Common Divisor (GCD) and Least Common Multiple (LCM) of two given numbers (provided by the user as input). Specify their asymptotic time and space complexities (using smallest upper bound).
- b) State whether the statements are true or false:
- i) The computing time needed for multiply and divide operation generally depends of the numbers/operands involved in the operation.
 - ii) In Data Structure arrays are always implemented by using consecutive memory locations.
- c) Write non-recursive and non-tail recursive algorithms (using *C* pseudocode) to find the factorial of a given number (provided by the user as input). Specify their asymptotic time and space complexities (using smallest upper bound).

(4+2+4)
